

DeToks

PART 2

Build Story & Technical Insight

이제 DeToks가 어떻게 “작업 흐름 컨트롤러”가 되었는지 설명합니다.



목차

큰 흐름만 먼저 공유하고, 이후 각 슬라이드에서 현재 위치를 표시합니다.

01**Phase 1 Review**

가설 검증과 방향 전환

02**Phase 2 Pivot**

설치·모델·Python 제거

03**Technical Walkthrough**

Cache · RAG · Budget 흐름

04**Use Cases & Roadmap**

활용 대상과 개선 방향

최초 계획

Phase 2 5/4 ~ 5/19

사용성 확장 및 추가 기능 구현

Phase 1 4/20 ~ 5/1

Token 절약을 위한 Core 기능 구현

Phase 1 Core 기능



Phase 1

가설 1

한국어를 영어로 바꾸면
토큰을 절약할 것이다

가설 2

작업을 분류·정제하면
LLM 부담이 줄어들 것이다

가설 3

전체 문맥 대신 요약 JSON을
저장·전달하면 줄어들 것이다

가설 1 · 입증

한국어를 영어로 바꾸면 입력 토큰을 절약할 수 있는가?

검증 데이터 · 로컬 LLM 벤치마킹

모델	샘플	성공률	추론 시간	토큰 감소
gemma-4-e2b	1047	99%	1.017s	21.177%
qwen3.5-0.8B	1047	87%	1.105s	25.699%
EXAONE-3.5-2.4B	1047	93%	1.468s	11.474%
supergemma-4-e4b	1047	100%	2.008s	21.174%

토큰 감소 범위

11.47% ~ 25.70%

가장 안정적

성공률 100%
supergemma-4-e4b

최대 절감

25.699%
qwen3.5-0.8B

Phase 1

가설 1 · 입증

한국어를 영어로 바꾸면
토큰을 절약할 것이다

번역 절감 효과 확인

가설 2

작업을 분류·정제하면
LLM 부담이 줄어들 것이다

가설 3

전체 문맥 대신 요약 JSON을
저장·전달하면 줄어들 것이다

가설 2 · 부분 입증

정제·압축은 효과가 있었지만, 분류 단계의 절감 효과는 검증하지 못했습니다.

PARTIAL

허깅페이스 153개 한국어 복합 프롬프트 데이터셋으로 입력 토큰 변화를 비교했습니다.

검증 결과 · 입력 토큰 비교

DeToks 미사용

3,784 tokens

DeToks 사용

2,424 tokens

번역 절감 30.243% · 압축 절감 6.507% · fallback 2건

입증된 부분

정제·압축은 입력 토큰 감소로 확인됨
→ 긴 문장을 줄이고 표현을 정리하는 효과

검증하지 못한 부분

분류가 별도 절감을 만드는지는 증명 실패
→ LLM이 이미 의도 분류를 잘 수행하던 영역

Phase 1

가설 1 · 입증

한국어를 영어로 바꾸면
토큰을 절약할 것이다

번역 절감 효과 확인

가설 2 · 부분 입증

작업을 분류·정제하면
LLM 부담이 줄어들 것이다

정제 · 압축 → 효과 있음

분류 → 절감 근거 없음

가설 3

전체 문맥 대신 요약 JSON을
저장·전달하면 줄어들 것이다

가설 3 · 입증

전체 문맥 대신 요약 JSON을 저장하면, 출력이 길수록 context 토큰 절감 폭이 커집니다.

RESULT

기존 raw output 전체를 다시 넣는 방식과 요약 JSON 저장 방식을 비교했습니다.

검증 결과 · task 출력 크기별 예상 절감

기존 task 출력	요약 JSON 크기	예상 절감
1KB	200~400B	60~80%
3KB	200~400B	85~93%
7KB	200~400B	93~97%

출력이 길수록 절감 폭이 커짐



저장되는 것

완료 task 결과를 요약 JSON과 상태 메타데이터로 저장

사용되는 것

다음 요청에서 필요한 문맥만 재구성해 context로 사용

참고

작업 길이, task 수, 보존 개수, 토큰라이저에 따라 변동

Phase 1

가설 1 · 입증

한국어를 영어로 바꾸면
토큰을 절약할 것이다

번역 절감 효과 확인 ✓

가설 2 · 부분 입증

작업을 분류·정제하면
LLM 부담이 줄어들 것이다

정제 · 압축 → 효과 있음 ✓

분류 → 절감 근거 없음 ✗

가설 3 · 입증

전체 문맥 대신 요약 JSON을
저장·전달하면 줄어들 것이다

출력이 길수록 절감 폭 증가 ✓

Phase 1 회고



이미 LLM이 잘 하고 있는 기능

LLM 자체의 작업 통제는 불가능

수정 계획

Phase 2 5/4 ~ 5/19

절약을 위한 추가 방법 모색
사용자 편의 및 추가 기능 구현

Phase 1 4/20 ~ 5/1

Token 절약을 위한 Core 기능 구현

Phase 2

사용자 편의

사용자 편의

BEFORE · 예전 실행 준비

여러 런타임과 서버를 직접 맞춰야 했습니다

```
$ npm install
$ python3.13 -m venv .venv
$ uv sync
$ export GGUF_MODEL_PATH=...
$ export API_BASE=http://localhost:...
$ ./llama-server -m model.gguf --port ...
$ npm run python-worker
```

설치보다 “환경 맞추기”와 “서버 띄우기”가 부담이었습니다.



통합

AFTER · 현재 실행 준비

Node 기반 설치 흐름으로 단순화

```
$ npm install @sorlros/detoks
```

로컬 추론 런타임을 **node-llama-cpp**로 통합
Python worker / llama-server를 따로 띄우지 않음

운영 부담 감소

Python 서버와 llama-server 프로세스 운영 제거

온보딩 단순화

Node만 준비되면 JS/네이티브 의존성 설치

범위 명확화

Adapter CLI와 GGUF 모델 준비는 별도일 수 있음

Llama Server 제거 및 번역 모델 교체

Gemma 실험 → Qwen GGUF 전환 → node-llama-cpp 통합

변경 사유 Gemma → Qwen: node-llama-cpp가 안정적으로 읽을 수 있는 GGUF(Q4_K_M) 모델로 전환

번역 품질 벤치마크

embedding cosine similarity로 의미 보존 정도를 자동 측정

Model	Success	Latency	Similarity	Fail	결론
Qwen3.5-4B GGUF	96.753%	5.620s	0.906	34	품질/속도 균형
Qwen3.5-2B GGUF	96.466%	5.779s	0.905	37	경량 후보
DeepSeek-R1 Qwen3-8B GGUF	96.848%	5.614s	0.908	33	최고 유사도

핵심: 번역 품질은 임베딩 유사도로 검증하고, 실행 구조는 Qwen GGUF + node-llama-cpp로 단순화했습니다.

Python worker 제거

BEFORE

Python ML worker 기반 압축

kompres-client.ts가 별도 Python 프로세스와 통신
Python 3.13 · uv · worker · 모델 환경 필요

Kompact worker ← Python process



AFTER

순수 TypeScript 규칙 기반 압축

node-compressor.ts에서 regex 치환 수행
kompres-client.ts는 wrapper로만 유지

compressTextWithKompres() → compressTextNode()

```
[/\bin order to\b/gi, "to"],  
[/\bso as to\b/gi, "to"],  
[/\bdue to the fact that\b/gi, "because"],  
[/\bat this point in time\b/gi, "now"],  
[/\bin the event that\b/gi, "if"],  
[/\bfor the purpose of\b/gi, "to"],  
[/\bwith regard to\b/gi, "regarding"],  
[/\bprior to\b/gi, "before"],  
[/\bsubsequent to\b/gi, "after"],  
[/\bat the present time\b/gi, "now"],  
[/\bin the near future\b/gi, "soon"],  
[/\ba large number of\b/gi, "many"],  
[/\ba number of\b/gi, "several"],  
[/\bin the process of\b/gi, "while"],
```

Phase 2

Token절감 방안

단순 압축에서 LLM 호출 최소화로

Before

입력 정제 및 압축



After

LLM 호출 최소화

현재 요청 받기 및 번역

USER INPUT

다음 주 현대차 IT팀 미팅 준비해야 해.
AI PC 관점에서 Intel vPro랑 Core Ultra 장점을 중심으로
고객 관심사 · 예상 질문 · 경쟁사 대비 포인트 · 15분 발표 구조를
만들어줘.

COMPILED PROMPT

Prepare a meeting plan for Hyundai Motor Group's IT team.
Focus on Intel vPro and Core Ultra benefits for AI PC adoption.
Include customer concerns, expected questions, competitor positioning,
and a 15-minute presentation structure.

현재 요청 받기 및 번역 Input & Translation

저장·조회해도 되는가?
Memory Policy

같은 요청을 이미 했나?
전체 요청 cache

요청을 작업으로 나누기
Task Graph

같은 작업을 이미 했나?
task 단위 cache

이 task에 참고할 자료 찾기
RAG 후보 검색

context를 넣을 가치가 있나?
Budget Gate

실제 AI CLI 호출
Adapter 실행

저장·조회해도 되는가?

이 단계는 답변 생성 전에, 현재 프로젝트에서 세션·캐시·RAG 저장소를 사용해도 되는지 먼저 확인합니다.

RESULT

Memory ON 세션 저장과 RAG 검색이 허용됩니다. 관련 과거 context는 이후 task 기준으로 자동 검색됩니다.

WHAT HAPPENS

- | | | |
|---|-----------------|--------------------------------------|
| 1 | 프로젝트 경로 확인 | 현재 폴더 기준으로 project id 생성 |
| 2 | Memory 설정 확인 | ON이면 session · cache · RAG 사용 가능 |
| 3 | 정책에 따라 사용 범위 결정 | OFF이면 cache · RAG · session 저장을 건너뛴다 |

현재 요청 받기 및 번역 Input & Translation

저장·조회해도 되는가?
Memory Policy

같은 요청을 이미 했나?
전체 요청 cache

요청을 작업으로 나누기
Task Graph

같은 작업을 이미 했나?
task 단위 cache

이 task에 참고할 자료 찾기
RAG 후보 검색

context를 넣을 가치가 있나?
Budget Gate

실제 AI CLI 호출
Adapter 실행

같은 요청을 이미 했나?

입력 전체 hash와 실행 환경을 확인해,
같은 요청이면 전체 결과를 재사용합니다.

COMPILED PROMPT

Prepare a meeting plan for Hyundai Motor Group's IT team.
Focus on Intel vPro and Core Ultra benefits for AI PC adoption.
Include customer concerns, expected questions, competitor positioning,
and a 15-minute presentation structure.

WHAT HAPPENS

- | | | |
|---|---------------|---------------------------------------|
| 1 | 입력 전체 hash 생성 | 현재 요청을 정규화해 session cache key 생성 |
| 2 | 실행 환경 확인 | adapter · model · git stamp 일치 여부 확인 |
| 3 | Hit / Miss 분기 | HIT: 전체 결과 재사용 · MISS: Task Graph로 이동 |

현재 요청 받기 및 번역
Input & Translation

저장·조회해도 되는가?
Memory Policy

같은 요청을 이미 했나?
전체 요청 cache

요청을 작업으로 나누기
Task Graph

같은 작업을 이미 했나?
task 단위 cache

이 task에 참고할 자료 찾기
RAG 후보 검색

context를 넣을 가치가 있나?
Budget Gate

실제 AI CLI 호출
Adapter 실행

요청을 작업으로 나누기

Compiled Prompt를 실행 가능한 task 단위로 분해합니다.

COMPILED PROMPT

Prepare a meeting plan for Hyundai Motor Group's IT team.
Focus on Intel vPro and Core Ultra benefits for AI PC adoption.
Include customer concerns, expected questions, competitor positioning,
and a 15-minute presentation structure.



TASK GRAPH RESULT

- TASK 1** Understand Hyundai IT team context
- TASK 2** Map vPro / Core Ultra benefits to AI PC adoption
- TASK 3** Prepare customer questions and competitor positioning
- TASK 4** Build a 15-minute presentation agenda

현재 요청 받기 및 번역
Input & Translation

저장·조회해도 되는가?
Memory Policy

같은 요청을 이미 했나?
전체 요청 cache

요청을 작업으로 나누기
Task Graph

같은 작업을 이미 했나?
task 단위 cache

이 task에 참고할 자료 찾기
RAG 후보 검색

context를 넣을 가치가 있나?
Budget Gate

실제 AI CLI 호출
Adapter 실행

같은 작업을 이미 했나?

TASKS FROM PREVIOUS STEP

01 Analyze Hyundai IT context
customer concerns · AI PC adoption

02 Summarize Intel messages
vPro security/management · Core Ultra NPU

03 Build objection points
purchase team · competitor response

04 Draft 15-minute agenda
meeting structure · expected Q&A

CACHE RESULT · HIT 2 / MISS 2

Task	Decision
01 Customer context analysis	HIT → 과거 회의 context 재사용
02 Intel sales message	MISS → 제품 메시지 새로 정리
03 Purchase / objection points	HIT → 구매 우려·반박 스크립트 재사용
04 15-minute agenda	MISS → 새 발표 구조 생성

HIT 2개는 adapter 실행을 생략하고, MISS 2개만 RAG · Budget · Adapter 단계로 내려갑니다.

현재 요청 받기 및 번역
Input & Translation

저장·조회해도 되는가?
Memory Policy

같은 요청을 이미 했나?
전체 요청 cache

요청을 작업으로 나누기
Task Graph

같은 작업을 이미 했나?
task 단위 cache

이 task에 참고할 자료 찾기
RAG 후보 검색

context를 넣을 가치가 있나?
Budget Gate

실제 AI CLI 호출
Adapter 실행

이 task에 참고할 자료 찾기

MISS task 2개만 검색하고, task별 상위 1개 snippet만 저장합니다.

SEARCH KEYS

Intel vPro · Core Ultra · Hyundai IT · TCO · competitor response

MISS task from F2 cache

02 Intel sales message
제품 메시지 새로 정리 필요

04 15-minute agenda
발표 구조 새로 생성 필요

Loaded RAG snippet

Product brief 1개
top-5 중 상위 1개만 load

Meeting note 1개
top-5 중 상위 1개만 load

현재 코드 기준 RAG load 흐름

1 hybridSearch
top-5 후보 검색

2 load top-1
상위 1개 snippet 로드

3 perTaskSnippets
task별 후보로 저장

4 Budget Gate
주입 여부 판단은 다음 단계

핵심: RAG는 “후보 수집” 단계입니다. 실제 context 주입은 Budget Gate가 결정합니다.

현재 요청 받기 및 번역
Input & Translation

저장.조회해도 되는가?
Memory Policy

같은 요청을 이미 했나?
전체 요청 cache

요청을 작업으로 나누기
Task Graph

같은 작업을 이미 했나?
task 단위 cache

이 task에 참고할 자료 찾기
RAG 후보 검색

context를 넣을 가치가 있나?
Budget Gate

실제 AI CLI 호출
Adapter 실행

context를 넣을 가치가 있나?

Budget Gate는 실행을 막지 않고, RAG context를 넣을지 말지만 결정합니다.

1. RAG에서 넘어온 자료

task당 1개 snippet

02 Product brief

Intel sales message

04 Meeting note

15-minute agenda

2. Budget Gate 확인

현재 코드 기준

1 토큰 예산

session / task cap

2 비용 대비 효과

break-even check

관련성 threshold는 아직 없음

3. 결정 결과

PASS → context 포함

BLOCK → context 제외

adapter 실행은 계속 진행

핵심: Budget Gate는 “실행 여부”가 아니라 “context 주입 여부”를 결정합니다.

현재 요청 받기 및 번역

Input & Translation

저장·조회해도 되는가?

Memory Policy

같은 요청을 이미 했나?

전체 요청 cache

요청을 작업으로 나누기

Task Graph

같은 작업을 이미 했나?

task 단위 cache

이 task에 참고할 자료 찾기

RAG 후보 검색

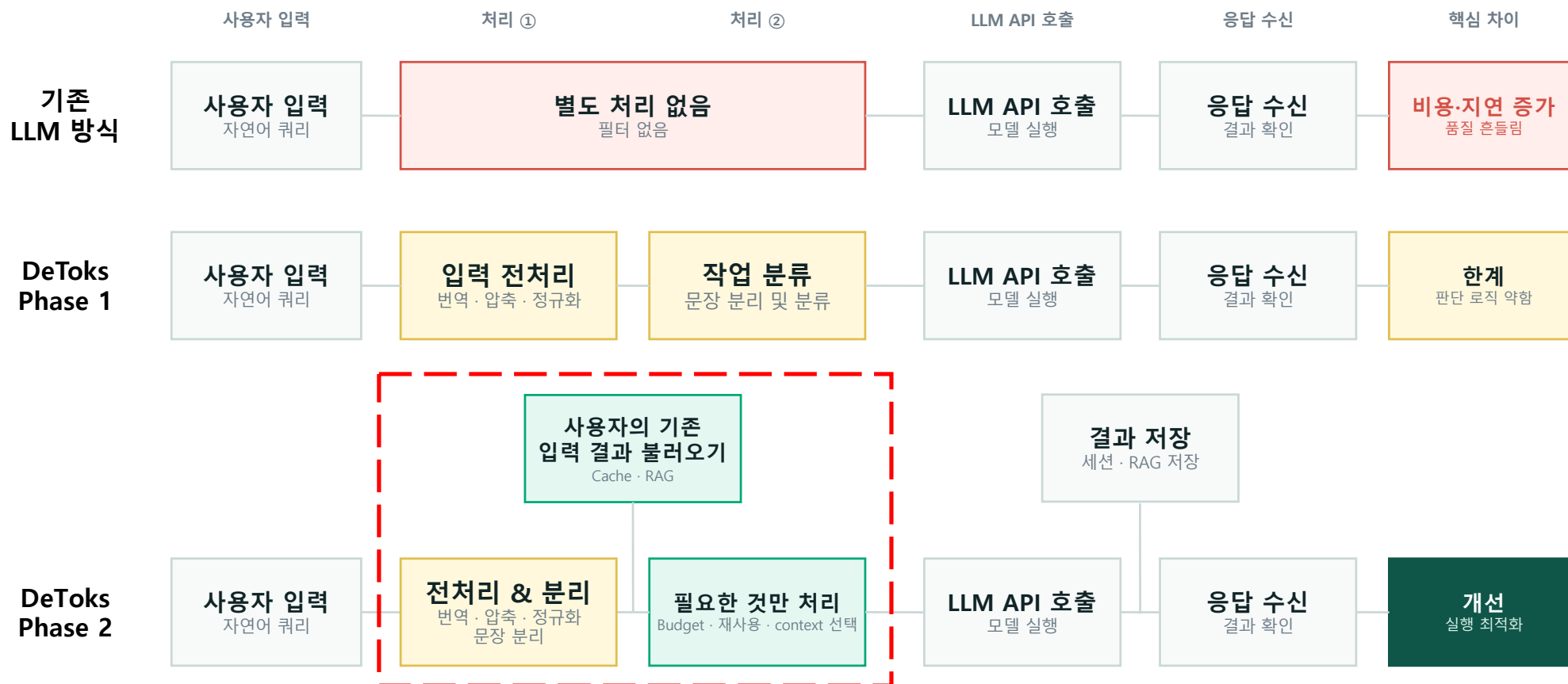
context를 넣을 가치가 있나?

Budget Gate

실제 AI CLI 호출

Adapter 실행

Method 비교 정리



이런 사람에게 유용합니다

AI CLI를 쓰지만, 반복 입력과 흐름 관리에 피로를 느끼는 사용자에게 맞습니다.

AI CLI를
자주 쓰는 개발자

반복 작업이
많은 팀

세션 관리가
필요한 프로젝트

토큰 비용을
관리할 조직

CLI는 낯설지만
AI 자동화에 관심 있는
IT 실무자

사용자는 “AI CLI를 더 쉽게, 더 오래, 더 반복 가능하게” 사용할 수 있습니다.

Known Issue

Phase 2 중간 전환 이후 남은 기술 리스크입니다.

01 RAG 인덱싱

성공 세션이 계속 쌓이면
DB가 비대해질 수 있음

02 RAG 검색

top-k 결과만 사용하면
낮은 관련성 context가 들어갈 수 있음

03 Cache Hit

의미는 같아도 표현이 다르면
재실행될 수 있음

04 테스트 시간

Phase 2 중간 pivot으로
실사용 검증 시간이 부족했음

핵심: 현재 기능은 동작하지만, 장기 사용성과 검색 품질 검증이 다음 과제입니다.

개선 방향

Known Issue를 품질 개선 항목으로 전환합니다.

01 RAG DB 관리

TTL 만료 삭제
Semantic deduplication

02 검색 품질 개선

Distance threshold
낮은 관련성 사전 필터링

03 Cache 고도화

Semantic cache lookup
표현이 달라도 유사 작업 매칭

04 지속 테스트

Known Issue 수정 후
실사용 테스트로 품질 개선

방향: 저장량을 줄이고, 검색 정확도를 높이고, 재사용 판단을 더 의미 기반으로 바꿉니다.

★ 랩 소개

한 줄 소개

CLI 환경에서 개발 프롬프트를 구조화하고, 토큰 효율화 방식을 실험하는 연구 모임입니다.

분야

공학

목표 결과물

오픈소스, 논문

DeToks LAB은 LLM CLI 환경에서 긴 개발 프롬프트를 더 효율적으로 다루기 위한 오픈소스 연구 모임입니다. 최근 개발자들은 Codex, Gemini CLI, Claude Code 등 다양한 AI 도구를 활용하지만, 긴 대화 흐름 속에서 불필요한 토큰 사용, 컨텍스트 손실, 반복 설명, 실패 시 복구 어려움 같은 문제가 자주 발생합니다.

본 모임은 이러한 문제를 해결하기 위해 프롬프트 구조화, 작업 단위 분해, 실험합니다. 단순히 기능을 개발하는 것에 그치지 않고, 일반 프롬프트 방식 복구 가능성 등을 정량적으로 검증하는 것을 목표로 합니다.

최종적으로는 DeToks CLI 고도화, 실험 리포트, 사용 가이드, 발표 자료,

알림톡 도착

모두의연구소
LAB 연구계획서 제출 완료

안녕하세요 윤지현님,
연구계획서 제출이 완료되었습니다.

계획서 검토는 1주일 정도 소요되며,
카카오톡으로 결과를 안내드립니다.

문의사항은 홈페이지를 이용해주세요.

내 연구계획서 보기

모두의연구소 바로가기

★ 운영 방식

모임은 주 1회 정기 오프라인 모임을 중심으로 운영합니다. 각 주차별로 2분담합니다. 모임 전에는 각자 맡은 기능 구현 또는 실험을 진행하고, 모임 설정을 진행합니다.

운영 방식은 크게 세 단계로 나뉩니다. 첫째, LLM CLI 사용 과정에서 발생 관리, fallback 기능 등을 설계하고 구현합니다. 셋째, 동일한 개발 과제를 실패율을 비교합니다.

모든 산출물은 GitHub 기반으로 관리하며, 주요 결과는 README, 실험 이해할 수 있도록 문서화와 재현 가능한 실험 환경을 중요하게 운영합니다.



DeToks LAB 랩장

CLI 환경에서 개발 프롬프트를 구조화하고, 토큰 효율화 방식을 실험하는 연구 모임입니다.

현재 모임이 진행 전입니다.

모집기간 : 2026.05.13 ~ 2026.05.30

모집기간 : 2026.06.01 ~ 2026.08.29

활동 연구원 수 3명 · 희망 모집 인원 3명



- 현재 모집 중이 아니므로 연구원 선발은 불가합니다.
- 추가 선발이 필요한 경우, [연구 정보]에서 모집 기간과 인원을 수정할 수 있습니다.

연구원 선발

연구원 목록

연구 정보

모임 정보

모임 기간

2026.06.01

모임 일시

월 화 수 목 금

10:30

모임 방식

온라인 오프라인

주기

매주 격주

장소

모두의연구소 강남캠퍼스

알림톡 도착

모두의연구소
LAB 개설 승인

안녕하세요 윤지현님 🎉
DeToks LAB의 랩장이 되신 것을 진심으로 축하드립니다!

✅ 모두연 슬랙 가입
아직 가입하지 않으셨다면, 슬랙에 가입해 주세요.

✅ 연구실 입장
연구실에 입장하여 LAB 운영 가이드를 숙지하고, 슬랙 채널과 노션 링크를 확인해주세요.

"Share Value, Grow Together"
랩장님의 즐거운 연구 여정을 모두의연구소가 함께하겠습니다.

문의사항은 홈페이지를 이용해주세요.

수익화 전략

Individual

OPEN SOURCE

Business

Business Fee

수익화 전략

Individual

Business

OPEN SOURCE

Business Fee

Buy Me a Coffee 

Try DeToks

```
$ npm install -g @sorlros/detoks  
$ detoks
```

Buy Me a Coffee 

